

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación



Sistema AGM Servicios Web

Alejandro Amador Lagunes

Alfredo Escudero Rivero

Hector Alejandro Mendoza Merino

Aaron Ulises Torres Corte

Joselyn Ramírez Lima

Johan Yuri Martínez García

Sistema AGM - Gestión Académica

1.1. Descripción General

GM Systems es una plataforma de gestión académica construida bajo una arquitectura de microservicios. El sistema tiene como objetivo apoyar procesos institucionales relacionados con autenticación de usuarios, administración de períodos y materias, control de docentes y alumnos, registro de asistencias, captura de calificaciones, generación de reportes y envío de notificaciones.

El proyecto fue diseñado como un ecosistema distribuido en el que cada módulo atiende una responsabilidad específica. Esta separación permite que los servicios puedan desarrollarse, probarse, desplegarse y escalarse de manera independiente. Además, el sistema utiliza contenedores Docker y Docker Compose para facilitar la ejecución local y la integración entre servicios.

El presente reporte describe de forma detallada la arquitectura, tecnologías utilizadas, comunicación interna, estructura del repositorio, responsabilidades de cada servicio, persistencia de datos, seguridad, pruebas, documentación de APIs e infraestructura del sistema. El núcleo de negocio de AGM-Systems comprende las siguientes áreas operativas:

- **Gestión de la Identidad y Seguridad (RBAC):** Autenticación centralizada y control de acceso basado en roles para Administradores, Docentes y Alumnos.
- **Planificación Académica:** Estructuración de periodos académicos, semestres, calendarios y cátedras.
- **Control de Materias y Oferta Educativa:** Catálogo de asignaturas con sus respectivos Números de Referencia de Curso (NRC) y asignaciones docentes.
- **Registro Centralizado de Calificaciones:** Captura y auditoría del historial evaluativo escolar.
- **Control de Asistencias Automatizado (QR):** Check-in dinámico mediante códigos QR temporales para prevenir fraudes en el aula.

- **Notificación Electrónica:** Envío de alertas automatizadas ante eventos críticos (bienvenidas, bajas, cierres de actas y restablecimiento de credenciales).
- **Reportería y Certificación:** Generación y descarga de concentrados de calificaciones e historiales de asistencia en formato PDF y Microsoft Excel

1.2 Alcance Técnico y Objetivos

Este manual técnico detalla la arquitectura de software, el diseño de la base de datos, las interfaces de comunicación, los contratos gRPC y el flujo de ejecución del sistema AGM-Systems. Tiene como objetivo principal servir de guía de referencia para ingenieros de desarrollo, DevOps, administradores de bases de datos y personal de soporte que requiera realizar tareas de mantenimiento, ampliación o despliegue del software.

Los objetivos fundamentales de la arquitectura implementada en AGM-Systems son:

1. **Alta Disponibilidad y Tolerancia a Fallos:** Garantizar que la indisponibilidad temporal de un módulo específico (por ejemplo, el módulo de notificaciones o de reportería) no afecte la operación de los servicios principales como la autenticación o la consulta de materias.
2. **Aislamiento de Dominios (Soberanía de Datos):** Adoptar un diseño desacoplado que impida el acceso cruzado a esquemas de bases de datos y obligue a la comunicación formal a través de APIs.
3. **Escalabilidad Horizontal Independiente:** Permitir que los servicios con alta demanda transaccional (tales como la toma de asistencia QR o el procesamiento de actas de calificaciones) se puedan escalar de forma individual en clústeres de servidores sin duplicar recursos innecesarios del sistema.
4. **Desarrollo Políglota y Evolutivo:** Mantener contratos de interfaz limpios que posibiliten el uso de diferentes lenguajes y frameworks según la idoneidad del caso de uso (p. ej., Django para CRUDs rápidos y FastAPI para APIs asíncronas de alto rendimiento).

1.3 Descripción General del sistema

AGM Systems está compuesto por varios microservicios especializados. Cada uno posee su propia lógica de negocio y, en la mayoría de los casos, su propia base de datos lógica.

La comunicación entre servicios se realiza mediante tres mecanismos principales:

- **REST:** usado para exponer funcionalidades al frontend y al API Gateway.
- **RabbitMQ con eventos:** usado para comunicación asíncrona entre servicios.
- **RabbitMQ RPC:** usado para consultas síncronas entre servicios sin acoplar directamente sus APIs HTTP.

El sistema también utiliza PostgreSQL como motor principal de persistencia, Redis para datos temporales de asistencia y SMTP4Dev como servidor SMTP de desarrollo para pruebas de correo.

2. Arquitectura de Software y Patrones de Diseño

2.1 Patrón Service-per-Database

En consonancia con las mejores prácticas para el desarrollo de sistemas distribuidos a gran escala, AGM-Systems implementa una arquitectura basada en microservicios bajo el patrón estructural Service-per-Database (Servicio por Base de Datos).

Este patrón aporta beneficios determinantes en el contexto escolar y empresarial del sistema:

- **Soberanía y Seguridad de Datos:** Ningún servicio puede realizar operaciones JOIN directas sobre tablas administradas por otro. Toda transferencia de información se realiza a través de llamadas de red controladas, garantizando que las reglas de negocio de cada dominio sean respetadas.
- **Resiliencia:** Si la base de datos de calificaciones experimenta un bloqueo o mantenimiento programado, los alumnos aún pueden ingresar al portal (vía ms-auth) y revisar su asistencia o calendario de clases. El fallo se contiene dentro del límite de su dominio (bounded context).

- Evolución del Stack: Es posible migrar la base de datos de un servicio específico a un motor diferente (por ejemplo, migrar ms-asistencias a una base de datos de series de tiempo NoSQL) sin requerir modificaciones en el esquema físico de los demás servicios.

2.2. Patrones de Comunicación Interna y Externa

El tráfico de datos dentro de la plataforma AGM-Systems está estrictamente clasificado en dos categorías operativas:

2.2.1. Comunicación Externa

Es la interacción entre la aplicación del cliente (Frontend SPA de Angular 20) y el sistema de backend. Se implementa mediante el protocolo REST/HTTP a través del API Gateway. El API Gateway centraliza aspectos transversales tales como:

- La validación y decodificación de tokens de seguridad JWT (emitidos por ms-auth).
- El enrutamiento y reenvío de peticiones HTTP REST hacia los microservicios internos.
- El formateo de cabeceras CORS y políticas de Rate Limiting.
- La agregación de respuestas complejas para optimizar el rendimiento del cliente.

2.2.2. Comunicación Interna

Es el intercambio de datos a alta velocidad entre los microservicios internos que coexisten dentro de la red privada del sistema. Se realiza mediante gRPC (Google Remote Procedure Call) sobre HTTP/2 y serialización binaria con Protocol Buffers (Protobuf).

- **Baja Latencia:** Al transferir datos binarios estructurados en lugar de texto plano JSON, se reduce el tamaño de los paquetes de red y el costo de serialización/deserialización.

- **Tipado Estricto:** Los contratos en archivos .proto funcionan como la fuente única de verdad, previniendo errores de integración en tiempo de ejecución.

2.3. Arquitectura de Concurrencia de Servicios

Los microservicios basados en FastAPI (como ms-docentes, ms-asistencias y ms-reportes) implementan un patrón de concurrencia de doble puerto. Al iniciar la aplicación, FastAPI se ejecuta de manera asíncrona sobre el servidor ASGI Uvicorn en un hilo principal para resolver las peticiones REST. Al mismo tiempo, se inicia un hilo secundario de tipo demonio (daemon thread) que aloja un servidor gRPC independiente en otro puerto físico. Esto permite a los servicios responder simultáneamente a peticiones REST (desde el API Gateway) y RPC (de microservicios adyacentes) usando una sola instancia de contenedor.

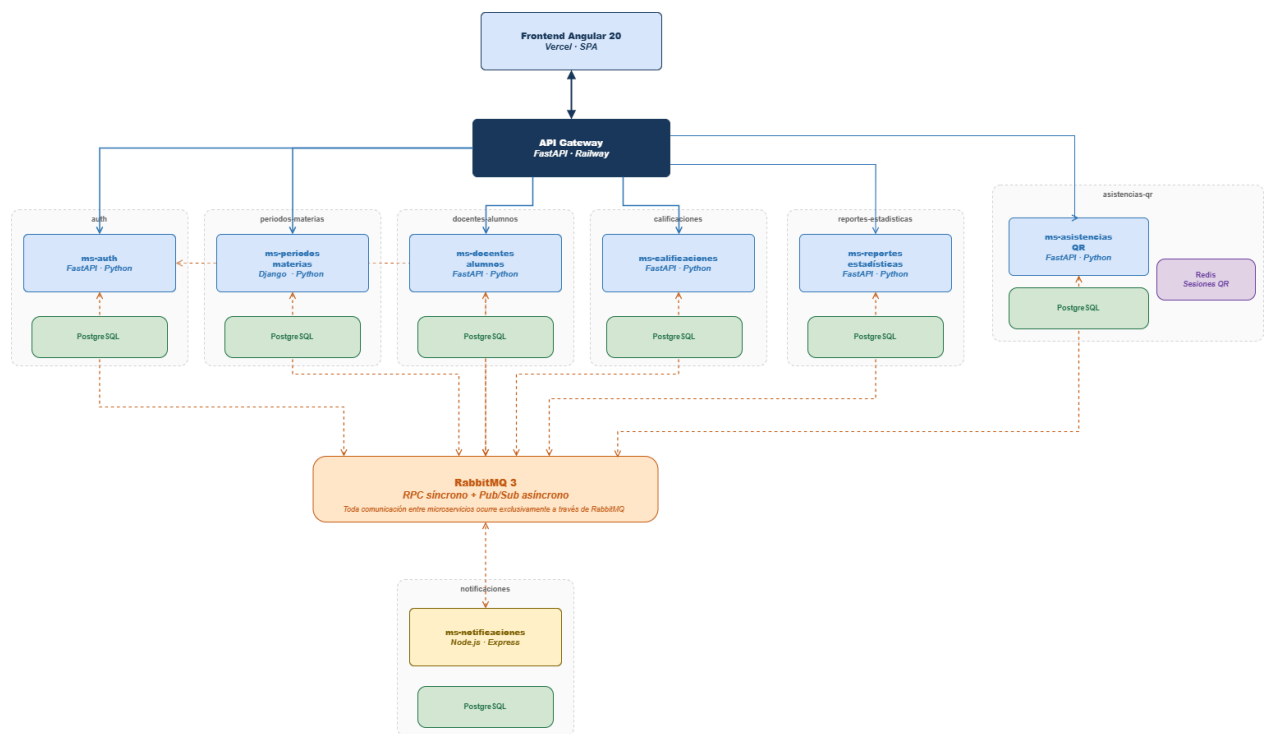


Figura 1. Arquitectura del sistema.

2.4. Ecosistema de Caché e Infraestructura de Soporte

- **Redis (Almacenamiento Ephemeral):** Utilizado por ms-asistencias para administrar el estado de las sesiones de asistencia activas y almacenar la caché de los tokens y códigos QR generados dinámicamente. Esto garantiza tiempos de respuesta inferiores a los 5 milisegundos en la validación del código QR del alumno.
- **SMTP4dev (Servidor de Desarrollo):** Herramienta sandbox integrada en la configuración de Docker Compose para capturar y previsualizar localmente las notificaciones de correo salientes sin requerir conexiones reales a servidores de correo empresariales.

2.5 API Gateway

El Gateway cuenta con una versión implementada en FastAPI y una plantilla de configuración Nginx. En FastAPI, se define un diccionario de servicios donde está definido cada uno de los puertos para los servicios:

```
SERVICES = {  
    "auth": {"host": os.getenv("MS_AUTH_HOST"), "port": 8000},  
    "periodos": {"host": os.getenv("MS_PERIODOS_HOST"), "port":  
        8000},  
    "docentes": {"host": os.getenv("MS_DOCENTES_HOST"), "port":  
        8003},  
    "calificaciones": {"host": os.getenv("MS_CALIFICACIONES_HOST"), "  
        port": 8004},  
    "asistencias": {"host": os.getenv("MS_ASISTENCIAS_HOST"), "port":  
        8005},  
    "reportes": {"host": os.getenv("MS_REPORTES_HOST"), "port":  
        8007},  
}
```

Figura 2. API Gateway.

3. Catálogo y Especificaciones de los Microservicios

3.1. ms-auth (Identidad y Seguridad)

Este servicio se encarga de la creación, autenticación y auditoría de usuarios del sistema, implementando un mecanismo de seguridad RBAC (Role-Based Access Control).

- Tecnologías: Python 3.11, FastAPI, SQLAlchemy, PostgreSQL, Passlib (encriptación con bcrypt).
- Estructura de Base de Datos: Tabla usuarios con campos id, email, password_hash y rol (representado por un enumerador de tres opciones: Administrador, Docente, Alumno).
- Cifrado: Usa bcrypt de 12 rondas de hashing para contraseñas.

3.2. ms-periodos-materias (Planificación y Oferta Académica)

Constituye el núcleo transaccional administrativo. Gestiona la oferta curricular del semestre escolar y las asignaturas vigentes.

- Tecnologías: Python 3.12, Django 5.2, Django REST Framework, gRPC (grpcio), PostgreSQL.
- Detalles de Implementación: Concentra las entidades Periodo y Materia en una sola base de datos lógica. Expone una API REST para administración básica y un servidor gRPC para que otros servicios validen la existencia de periodos y materias.
- Regla de negocio crítica: El guardado de periodos implementa una transacción atómica donde, si se define un periodo como Activo, se desactivan automáticamente todos los periodos anteriores.

3.3. ms-docentes (Catálogo y Programación Académica)

Maneja el padrón de profesores y el listado oficial de alumnos inscritos en las materias.

- Tecnologías: FastAPI, SQLAlchemy, PostgreSQL, Python-slim en Docker.
- Método de Carga de Datos: Soporta flujos de importación masiva a partir de archivos de programación académica en formato PDF y listas de asistencia en Microsoft Excel.
- Servicios gRPC expuestos: Permite consultar información detallada de alumnos y docentes individuales por ID, resolver los alumnos inscritos bajo un NRC específico y validar si un alumno pertenece o no a una materia.

3.4. ms-asistencias (Registro QR)

Servicio de alta velocidad responsable del registro de la asistencia diaria en las aulas.

- Tecnologías: FastAPI, SQLAlchemy, PostgreSQL, Redis (Caché).
- Flujo QR: Un docente inicia una sesión de asistencia desde su dispositivo móvil. El sistema genera un código QR firmado en Redis con un tiempo de expiración corto (p. ej. 30 segundos). El alumno escanea el QR y envía su identificador de inscripción; el servicio valida el QR contra Redis de forma instantánea. Si es válido, inserta de manera asíncrona un registro en PostgreSQL con estado Presente o Retardo según la hora límite definida.

3.5. ms-notificaciones (Alertas por Correo)

Administra las notificaciones salientes del sistema mediante colas de correo.

- Tecnologías: Node.js (CommonJS), @grpc/grpc-js, @grpc/proto-loader, Express, Nodemailer, pg (PostgreSQL Client).
- Flujo de Operación: Este servicio es puramente gRPC (consumidor de red interna). Recibe peticiones de envío del sistema y, en segundo plano, se conecta a los clientes gRPC de ms-periodos-materias y ms-docentes para reconstruir los nombres y correos reales de alumnos o docentes. Posteriormente, envía el email estructurado

en formato HTML a través del transporte SMTP. Por cada envío, guarda un registro histórico en la tabla `historial_correos` de su base de datos.

3.6. ms-reportes (Agregador y Generación de Archivos)

Compila y genera la documentación física de calificaciones e historial escolar.

- Tecnologías: FastAPI, ReportLab (generación de PDF), openpyxl (generación de archivos Excel de hoja de cálculo), python-jose (JWT validation), gRPC.
- Integración: Consolida datos JSON externos enviados en la llamada RPC para incrustar tablas estilizadas, membretes institucionales, firmas digitales y promedios generales en certificados.

3.7 Stack Tecnológico

El sistema utiliza diferentes tecnologías seleccionadas de acuerdo con la responsabilidad de cada componente.

3.7.1. Backend

- FastAPI: utilizado en varios microservicios por su rendimiento, tipado, integración con Pydantic y generación automática de documentación OpenAPI.
- Django REST Framework: utilizado en el servicio de periodos y materias para aprovechar el ecosistema de Django, sus modelos, validaciones y ViewSets.
- Node.js: utilizado en el microservicio de notificaciones, ideal para operaciones de entrada/salida no bloqueantes, como envío de correos y consumo de colas.

3.7.2. Persistencia

- PostgreSQL: motor relacional principal. Se usa para almacenar usuarios, materias, docentes, alumnos, asistencias, calificaciones, reportes e historial de correos.
- Redis: almacén en memoria utilizado por el servicio de asistencias para manejar tokens, QR temporales y sesiones efímeras.

3.7.3. Mensajería

- RabbitMQ: broker de mensajes usado para comunicación asíncrona mediante eventos y comunicación síncrona mediante patrón RPC.

3.7.4. Infraestructura

- Docker: empaquetado de servicios en contenedores.
- Docker Compose: orquestación local de microservicios, bases de datos, Redis, RabbitMQ y SMTP4Dev.
- SMTP4Dev: servidor SMTP de desarrollo usado para probar correos electrónicos sin depender de un proveedor externo.

3.7.5. Herramientas de Documentación y Pruebas

- Postman: colecciones de endpoints por microservicio y flujos completos del sistema.
- Pytest / pruebas funcionales: pruebas ubicadas en el directorio tests/.
- OpenAPI: documentación automática generada por FastAPI.

4. Diseño y Modelo Físico de Base de Datos

El motor principal de persistencia relacional es PostgreSQL (versión 15 en contenedor Docker). En el entorno local, se inicializan de forma lógica las bases de datos asignadas a cada microservicio para mantener el aislamiento transaccional del patrón Service-per-Database.

4.1. Estructura SQL de Inicialización (init.sql)

El script de inicialización de la infraestructura SQL define las bases de datos lógicas y la estructura inicial para los microservicios de notificaciones y docentes:

4.2. Estructura de Índices y Optimización de Consultas

Como se aprecia en el listado, se declaran explícitamente índices sobre campos críticos para

garantizar una respuesta rápida en consultas de alto tráfico escolar:

- `ix_docentes_nombre` e `ix_docentes_email`: Optimizan la resolución rápida del nombre de profesores para el armado de correos de baja de alumnos o para la visualización de perfiles en el API Gateway.
- `ix_alumnos_nrc` e `ix_alumnos_matricula`: Optimizan las búsquedas e inserciones masivas que ocurren durante el procesamiento de listas de alumnos por NRC y validan que el alumno pertenezca al curso al escanear la asistencia QR.

4.3. Modelos Adicionales de Microservicios

A continuación se describen los esquemas de bases de datos generados mediante los ORMs correspondientes a nivel de microservicio:

5. Despliegue y Orquestación con Docker Compose

El entorno de ejecución de AGM-Systems está completamente orquestado y empaquetado para correr dentro de contenedores aislados. Esto asegura que no existan conflictos de dependencias entre Python 3.11, Python 3.12 y Node.js. A continuación, se detalla la configuración del archivo docker-compose empleado para instanciar la suite completa:

5.1 Configuración de Archivos de Entorno (.env)

Para cada microservicio, se requiere crear un archivo `.env` basado en su plantilla `.env.example` correspondiente. Estos archivos definen los parámetros de conexión dinámicos y claves de cifrado. Al desplegar la suite completa mediante Docker Compose, las variables internas de conexión de red se resuelven utilizando los DNS internos del Docker Engine (como `postgres-db`, `redis-db` o `smtp4dev`). En caso de ejecutar un servicio de manera local y fuera de Docker (para debugging), la variable `DB_HOST` o `DATABASE_URL` debe apuntar a `localhost`.

6.2 Instrucciones de Despliegue y Comandos de Consola

Para levantar la infraestructura de microservicios completa en modo productivo o desarrollo, siga las siguientes instrucciones en la terminal de comandos:

- Para construir y ejecutar el entorno completo:

```
docker-compose up --build
```

- Para acceder al gateway:

```
http://localhost
```

- Para acceder a RabbitMQ Management

```
http://localhost:15672
```

7. Persistencia de Datos

El sistema utiliza el patrón “base de datos por servicio” a nivel lógico. En el entorno local se ejecuta un único contenedor PostgreSQL, pero se crean varias bases de datos independientes.

7.1. Bases de datos creadas

El script `infrastructure/init.sql` crea las siguientes bases de datos:

- `agm_auth_db`
- `agm_notificaciones_db`
- `attendance_db`
- `agm_docentes_db`
- `agm_periodos_materias_db`
- `reports_db`

- agm_calificaciones_db

7.2. Ventajas

- Cada servicio mantiene control sobre sus propios datos.
- Se evita que un servicio modifique directamente las tablas de otro.
- Facilita cambios independientes de esquema.
- Permite aislar fallos o migraciones.

7.3 Redis

Redis se utiliza exclusivamente como almacenamiento temporal para la lógica de asistencias. Es útil en este contexto porque:

- Los tokens QR son efímeros.
- Se necesita lectura rápida.
- La expiración automática simplifica el manejo de sesiones.

8. Seguridad

La seguridad del sistema se basa principalmente en autenticación JWT, hash de contraseñas y validación de roles.

8.1. JWT

El servicio de autenticación genera tokens JWT después de validar las credenciales del usuario. Estos tokens son enviados por el cliente en solicitudes posteriores.

Un flujo típico es:

1. El usuario envía email y contraseña al endpoint de login.
2. ms-auth valida las credenciales.
3. Se genera un JWT firmado con una clave secreta.

4. El frontend almacena temporalmente el token.
5. Las solicitudes protegidas incluyen el token en el encabezado Authorization.
6. Los servicios validan el token localmente o mediante RPC con ms-auth.

8.2. Contraseñas

Las contraseñas no se almacenan en texto plano. Se utiliza Bcrypt mediante Passlib para generar hashes seguros.

8.3. Roles

El modelo de usuario define tres roles:

- Administrador.
- Docente.
- Alumno.

Estos roles permiten controlar permisos y restringir funcionalidades según el tipo de usuario.

8.4. Recuperación de contraseña

El sistema incluye una tabla de tokens de recuperación. Cada token tiene:

- Usuario asociado.
- Hash del token.
- Fecha de expiración.
- Fecha de uso.
- Fecha de creación.

Esto permite implementar enlaces de recuperación de un solo uso

9. Manejo de Archivos

El proyecto incluye funcionalidades de importación y exportación de archivos.

9.1. Importación

- PDF: utilizado para importar docentes, alumnos o materias desde documentos académicos.
- Excel: utilizado para importar alumnos y calificaciones.

Librerías utilizadas:

- pdfplumber
- pypdf
- pandas
- openpyxl

9.2. Exportación

El servicio de reportes genera documentos en:

- PDF: mediante ReportLab.
- Excel: mediante OpenPyXL.

10. Postman y Pruebas Manuales

El directorio postman/ contiene colecciones separadas por microservicio:

- MS-Auth.
- MS-PeriodosMaterias.
- MS-Docentes.
- MS-Asistencias.
- MS-Calificaciones.

- MS-Reportes.
- MS-Notificaciones.

11. Full-System-Flow.

Estas colecciones permiten validar endpoints de forma manual y ejecutar flujos completos, como:

1. Login de docente.
2. Inicio de sesión de asistencia.
3. Login de alumno.
4. Generación de token QR.
5. Registro de asistencia.
6. Consulta de asistencias del día.

12. Decisiones de Diseño

12.1. Uso de Microservicios

Se eligió una arquitectura de microservicios para separar responsabilidades y permitir evolución independiente. Cada dominio académico tiene reglas particulares, por lo que dividir el sistema evita concentrar toda la lógica en una aplicación monolítica.

12.2. Uso de RabbitMQ

RabbitMQ permite desacoplar servicios. La combinación de eventos y RPC ofrece flexibilidad:

- Los eventos sirven para acciones que no requieren respuesta inmediata.
- RPC sirve para consultas que sí necesitan una respuesta.

12.3. Uso de Redis

Redis es adecuado para tokens QR porque estos datos tienen vida corta y requieren lectura rápida.

12.4. Uso de PostgreSQL

PostgreSQL ofrece consistencia, integridad relacional, soporte para índices y compatibilidad amplia con ORMs como SQLAlchemy y Django ORM.

12.5. Uso de Docker Compose

Docker Compose simplifica la ejecución del sistema completo. Permite levantar servicios, bases de datos y herramientas auxiliares con un solo comando.

13. Riesgos y Áreas de Mejora

Aunque el sistema cuenta con una arquitectura sólida, existen áreas que podrían fortalecerse:

- Homologar versiones de FastAPI, SQLAlchemy y Pydantic entre microservicios.
- Centralizar configuración sensible mediante un gestor de secretos.
- Agregar observabilidad con logs estructurados, métricas y trazas distribuidas.
- Implementar migraciones formales en todos los servicios, por ejemplo Alembic paraFastAPI.
- Agregar pruebas de contrato entre servicios.
- Fortalecer políticas de autorización por rol en todos los endpoints.
- Documentar formalmente los eventos RabbitMQ y sus esquemas.
- Agregar tolerancia a fallos y reintentos para consumidores de cola.
- Separar credenciales de desarrollo y producción.
- Automatizar CI/CD para pruebas, construcción de imágenes y despliegue.

14. Conclusiones

AGM Systems representa una solución académica integral basada en microservicios. La plataforma integra múltiples dominios funcionales: autenticación, periodos, materias, docentes, alumnos, asistencias, calificaciones, reportes y notificaciones. El diseño permite que cada módulo tenga independencia tecnológica y funcional. FastAPI se utiliza para servicios ligeros y de alto rendimiento; Django REST Framework se aprovecha para la administración académica estructurada; Node.js se usa para el procesamiento asíncrono de notificaciones; PostgreSQL garantiza persistencia relacional; Redis resuelve el manejo de tokens temporales; RabbitMQ permite comunicación desacoplada; y Docker Compose facilita la ejecución del entorno completo. En conjunto, el proyecto demuestra una arquitectura moderna orientada a servicios, con una separación clara de responsabilidades, mecanismos de comunicación adecuados y una base preparada para crecer hacia escenarios más robustos de despliegue, monitoreo, pruebas y escalabilidad.